

Cálculo e Gradiente Descendente

A ladeira que todo modelo desce para aprender

Toda vez que um modelo "treina", ele está resolvendo um problema de otimização: encontrar os parâmetros que minimizam uma função de perda. Este material constrói derivada, gradiente e Hessiana a partir da ideia de aproximação linear local, explica por que convexidade é a propriedade que separa "sempre converge" de "pode travar", e implementa gradiente descendente e suas variantes do zero — o algoritmo que treina desde uma regressão linear até um Transformer.

Nível: Intermediário — requer os módulos anteriores deste tema

Pre-requisitos: Vetores, Matrizes e Projeções; Decomposições — SVD, Autovalores e PCA

Carga sugerida: 8 a 10 horas (leitura + 3 notebooks)

Notebooks: 01-derivadas-e-gradientes · 02-gradiente-descendente · 03-convexidade-e-condicionamento · 99-exercicios

Autoria: Material do curso de ML & DL

Versao: 1.0

Sumário

1. Por que este módulo existe	3
1.1 O que você vai conseguir fazer ao final	3
2. Derivada: a taxa de variação, e sua melhor aproximação linear	4
2.1 Derivada numérica: quando não há fórmula fechada	4
3. Gradiente: a derivada em várias dimensões	5
3.1 Gradiente de funções comuns em machine learning	5
4. Hessiana: a curvatura	6
5. Convexidade: a propriedade que separa "sempre dá certo" de "depende"	7
5.1 Reconhecendo convexidade	7
6. Gradiente descendente: o algoritmo	8
6.1 O papel decisivo da taxa de aprendizado	8
6.2 Gradiente descendente estocástico (SGD)	8
7. Variantes que aceleram e estabilizam	9
7.1 Momentum: memória da direção	9
7.2 Adagrad e RMSprop: taxa de aprendizado por parâmetro	9
7.3 Adam: o padrão de mercado	9
8. Condicionamento: por que alguns problemas são teimosos	10
9. Erros que custam caro — checklist	11
10. Para ir além	12

1. Por que este módulo existe

"Treinar um modelo" é uma frase que esconde uma ideia simples: definir um número que mede o quão errado o modelo está (a função de perda) e mover os parâmetros na direção que faz esse número diminuir. Repetir esse movimento milhares de vezes é, literalmente, todo o "aprendizado" de uma regressão linear, uma árvore de gradient boosting ou uma rede neural de bilhões de parâmetros. Este módulo constrói o vocabulário — derivada, gradiente, Hessiana, convexidade — e o algoritmo — gradiente descendente — que sustentam essa frase.

ANALOGIA

Imagine estar em uma trilha de montanha, no meio da neblina, e precisar descer até o vale mais rápido possível. Você não vê o mapa inteiro — só sente, sob os pés, para que lado o chão desce mais íngreme *agora*. Dar um passo nessa direção, sentir de novo, dar outro passo: isso é gradiente descendente. O gradiente é exatamente essa sensação sob os pés, formalizada.

1.1 O que você vai conseguir fazer ao final

- Explicar o que uma derivada realmente mede, e generalizar para várias dimensões via o gradiente.
 - Ler uma Hessiana e saber o que ela diz sobre a forma local da função.
 - Distinguir uma função convexa de uma não-convexa e explicar por que isso determina se a otimização é "fácil" ou "pode travar".
 - Implementar gradiente descendente do zero e diagnosticar uma taxa de aprendizado boa, grande demais ou pequena demais.
 - Explicar o que momentum, Adagrad, RMSprop e Adam corrigem, e por que o número de condição da Hessiana é o vilão comum a todos os problemas de convergência.
-

2. Derivada: a taxa de variação, e sua melhor aproximação linear

A derivada de f em x é o limite:

$$f'(x) = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}$$

Duas leituras, ambas úteis:

- 1 **Inclinação.** $f'(x)$ é a inclinação da reta tangente ao gráfico de f em x — quão rápido f muda perto daquele ponto.
- 2 **Melhor aproximação linear.** Perto de x , $f(x+h) \approx f(x) + f'(x)h$. Essa é a ideia mais importante do cálculo aplicado a otimização: toda função suave, olhada de perto o suficiente, **parece uma reta** — e otimizar é basicamente usar essa reta local para decidir para onde andar.

NOTA

Na prática de machine learning, quase nunca se deriva à mão além de exemplos didáticos. Bibliotecas de **diferenciação automática** (autograd, PyTorch, TensorFlow, JAX) calculam derivadas exatas de qualquer composição de operações, aplicando a regra da cadeia mecanicamente. Este módulo constrói a intuição que torna esse mecanismo compreensível — o assunto do módulo de Backpropagation, no tema de Deep Learning.

2.1 Derivada numérica: quando não há fórmula fechada

Quando a fórmula de f' não está disponível (ou para checar se uma derivação manual está certa), a diferença finita central aproxima a derivada:

$$f'(x) \approx \frac{f(x+h) - f(x-h)}{2h}$$

ARMADILHA COMUM

A diferença finita **central** (usando $x+h$ e $x-h$) tem erro da ordem de h^2 ; a diferença **progressiva** ($f(x+h) - f(x)$, dividido por h) tem erro de ordem h — dez vezes menos precisa para o mesmo h . E h não pode ser arbitrariamente pequeno: abaixo de $\sim 10^{-8}$, o erro de arredondamento de ponto flutuante domina e a aproximação piora de novo. Existe um h ótimo, nem grande nem minúsculo — testar sensibilidade a h é a forma padrão de verificar (**gradient checking**) uma derivada implementada à mão.

3. Gradiente: a derivada em várias dimensões

Para $f: \mathbb{R}^n \rightarrow \mathbb{R}$, o **gradiente** é o vetor de derivadas parciais:

$$\nabla f(x) = \left(\frac{\partial f}{\partial x_1}, \dots, \frac{\partial f}{\partial x_n} \right)$$

Cada derivada parcial mede como f muda quando **só** aquela coordenada se move, mantendo as outras fixas.

FORMULARIO

O **gradiente aponta na direção de maior crescimento** de f , e $-\nabla f(x)$ aponta na direção de **maior decrescimento**. Essa é a propriedade que faz o gradiente descendente funcionar: dado um ponto, mover-se contra o gradiente é a forma mais eficiente de diminuir f localmente.

A mesma ideia de aproximação linear se generaliza: perto de x ,

$$f(x + h) \approx f(x) + \nabla f(x)^\top h$$

— e essa aproximação (o plano tangente) é a base de todo algoritmo de primeira ordem, incluindo o gradiente descendente.

3.1 Gradiente de funções comuns em machine learning

Função	Gradiente	Onde aparece
$f(\beta) = \ X\beta - y\ ^2$	$\nabla f = 2X^\top (X\beta - y)$	mínimos quadrados
$f(w) = \frac{1}{2} \ w\ ^2$	$\nabla f = w$	regularização L2 (Ridge)
$f(w) = -\sum_i [y_i \log \hat{y}_i + (1 - y_i) \log(1 - \hat{y}_i)]$	Envolve $\hat{y}_i - y_i$	entropia cruzada, regressão logística

NO MERCADO

O gradiente da soma de quadrados, $2X^\top (X\beta - y)$, é a razão pela qual a fórmula de mínimos quadrados do módulo 1 ($\hat{\beta} = (X^\top X)^{-1} X^\top y$) existe: ela é exatamente o ponto onde esse gradiente é zero. Toda solução fechada de um problema convexo é, por trás, "resolva $\nabla f = 0$ " — quando isso tem fórmula fechada, você não precisa de gradiente descendente. Regressão logística, SVM e redes neurais não têm fórmula fechada: por isso usam gradiente descendente.

4. Hessiana: a curvatura

A **Hessiana** é a matriz de segundas derivadas parciais:

$$H_{ij} = \frac{\partial^2 f}{\partial x_i \partial x_j}$$

Se o gradiente diz "para que lado a função desce", a Hessiana diz "o quanto a inclinação está mudando" — ou seja, a **curvatura**. Uma Hessiana é sempre simétrica (para funções suaves — teorema de Schwarz), então tudo o que o módulo anterior construiu sobre matrizes simétricas (autovalores reais, autovetores ortogonais, teorema espectral) se aplica diretamente a ela.

DEFINIÇÃO

Condições de otimalidade de segunda ordem, num ponto crítico ($\nabla f(x^*) = 0$):

- Se todos os autovalores da Hessiana são positivos (H é **positiva definida**), x^* é um **mínimo local**.
- Se todos são negativos, x^* é um **máximo local**.
- Se há autovalores de sinais diferentes, x^* é um **ponto de sela** — nem mínimo nem máximo, mas um ponto crítico do gradiente ainda assim.

ARMADILHA COMUM

Em dimensões altas (como uma rede neural com milhões de parâmetros), pontos de sela são **muito mais comuns** que mínimos locais genuínos. A intuição de otimização em 1D — "o gradiente descendente pode ficar preso num mínimo local ruim" — é menos relevante em alta dimensão do que a intuição de "o gradiente descendente pode ficar lento perto de uma sela", porque o gradiente fica pequeno ali sem que o ponto seja de fato um mínimo.

5. Convexidade: a propriedade que separa "sempre dá certo" de "depende"

Definição geométrica. Uma função é **convexa** se o segmento de reta entre dois pontos quaisquer do seu gráfico fica **acima ou sobre** o gráfico:

$$f(\theta x + (1 - \theta)y) \leq \theta f(x) + (1 - \theta)f(y), \quad \forall \theta \in [0, 1]$$

ANALOGIA

Uma função convexa é uma tigela. Uma não-convexa pode ter vales, morros e platôs. Rolar uma bola numa tigela sempre a leva ao fundo, não importa onde ela começa. Rolar uma bola numa paisagem cheia de vales pode prendê-la em um vale que não é o mais fundo de todos.

FORMULARIO

Por que convexidade importa tanto: numa função convexa, **todo mínimo local é o mínimo global**. Gradiente descendente com taxa de aprendizado adequada tem garantia teórica de convergência. Numa função não-convexa, essa garantia desaparece — o algoritmo pode convergir para um mínimo local diferente dependendo de onde começou.

5.1 Reconhecendo convexidade

- Se a segunda derivada existe: f é convexa em um intervalo se $f''(x) \geq 0$

em todo ponto dele (em várias dimensões: Hessiana positiva semidefinida em todo o domínio).

- Soma de funções convexas é convexa.
- $\|x\|^2$, $\|x\|_1$, e^x , e a função de perda dos mínimos quadrados

($\|x - y\|^2$, quadrática em β) são convexas.

- Entropia cruzada de regressão logística é convexa em relação aos parâmetros.
- A perda de uma rede neural com camadas ocultas não-lineares não é

convexa — a composição de funções não-lineares destrói a propriedade, mesmo que cada peça isolada seja simples.

NO MERCADO

É por isso que regressão linear e regressão logística têm garantia de encontrar o ótimo global (a perda é convexa), enquanto redes neurais profundas não têm garantia nenhuma — e ainda assim funcionam bem na prática. Entender por que isso acontece (superfícies de perda de redes neurais têm estrutura muito mais benigna do que "não-convexo" sugere à primeira vista) é um tópico de pesquisa ativo, tratado com mais profundidade no tema de Deep Learning.

6. Gradiente descendente: o algoritmo

$$x_{t+1} = x_t - \eta \nabla f(x_t)$$

onde η (a **taxa de aprendizado**) controla o tamanho do passo.

FORMULARIO

A cada passo: (1) calcule o gradiente no ponto atual; (2) dê um passo na direção oposta ao gradiente, de tamanho η ; (3) repita até o gradiente ficar próximo de zero (convergência) ou até um número máximo de iterações.

6.1 O papel decisivo da taxa de aprendizado

η	Comportamento
Muito pequena	Converge, mas devagar — muitas iterações desperdiçadas
Adequada	Converge rápido e de forma estável
Muito grande	Oscila, pode divergir (o passo "pula" para o outro lado do vale e piora)

ARMADILHA COMUM

Um sintoma comum de taxa de aprendizado grande demais: a perda **cresce** ou oscila violentamente entre iterações, em vez de cair de forma monotônica ou suavemente decrescente. O primeiro instinto ao ver isso deveria ser diminuir η , não trocar de algoritmo.

6.2 Gradiente descendente estocástico (SGD)

Em machine learning, f costuma ser uma soma sobre n exemplos: $f(\theta) = \frac{1}{n} \sum_i \ell_i(\theta)$. Calcular o gradiente exato exige passar por todos os n exemplos a cada passo — caro quando n é grande. O **SGD** aproxima o gradiente usando só um exemplo (ou um pequeno lote, *mini-batch*) por vez:

$$\theta_{t+1} = \theta_t - \eta \nabla \ell_i(\theta_t)$$

O gradiente estimado é **ruidoso** (não é o gradiente exato), mas é muito mais barato de calcular, e o ruído às vezes até ajuda a escapar de mínimos locais rasos e pontos de sela. É o algoritmo que treina praticamente todo modelo de deep learning em produção — sempre com mini-batches, nunca com o dataset inteiro por passo.

7. Variantes que aceleram e estabilizam

7.1 Momentum: memória da direção

$$v_{t+1} = \mu v_t + \nabla f(x_t), \quad x_{t+1} = x_t - \eta v_{t+1}$$

Acumula uma média móvel dos gradientes passados. Intuição física: uma bola rolando ganha inércia — atravessa pequenas ondulações do terreno em vez de travar nelas, e acelera em direções consistentes.

7.2 Adagrad e RMSprop: taxa de aprendizado por parâmetro

Adagrad divide a taxa de aprendizado de cada parâmetro pela raiz da soma dos quadrados dos gradientes passados **daquele parâmetro** — parâmetros com gradientes historicamente grandes recebem passos menores, e vice-versa. RMSprop conserta o principal defeito do Adagrad (a taxa efetiva cai a zero com o tempo, porque a soma só cresce) trocando a soma por uma **média móvel exponencial**, que "esquece" gradientes antigos.

7.3 Adam: o padrão de mercado

Adam combina momentum (média móvel do gradiente) com a ideia do RMSprop (média móvel do gradiente ao quadrado, para escalar a taxa por parâmetro):

$$m_{t+1} = \beta_1 m_t + (1 - \beta_1) \nabla f(x_t), \quad v_{t+1} = \beta_2 v_t + (1 - \beta_2) \nabla f(x_t)^2$$
$$x_{t+1} = x_t - \eta \frac{\hat{m}_{t+1}}{\sqrt{\hat{v}_{t+1} + \epsilon}}$$

(com $\hat{m}$, $\hat{v}$ sendo versões corrigidas de viés de m , v — o detalhe fica para o tema de Deep Learning). Na prática, Adam é o otimizador padrão para treinar redes neurais: converge rápido, é razoavelmente robusto à escolha de η , e lida bem com gradientes esparsos ou de magnitudes muito diferentes entre parâmetros.

NO MERCADO

Isso não significa que Adam é sempre a melhor escolha. SGD com momentum bem ajustado às vezes generaliza melhor em visão computacional; Adam é quase universal em NLP e Transformers. A escolha do otimizador é, na prática, mais uma questão empírica do domínio do que uma verdade matemática única.

8. Condicionamento: por que alguns problemas são teimosos

Para uma função quadrática $f(x) = \frac{1}{2}x^T Ax$, a Hessiana é A (constante). O **número de condição** de A — a razão entre o maior e o menor autovalor, $\kappa(A) = \lambda_{\max}/\lambda_{\min}$ — controla diretamente o comportamento do gradiente descendente:

ARMADILHA COMUM

Quando κ é grande, as curvas de nível de f são elipses muito alongadas (a mesma imagem geométrica do módulo anterior). O gradiente aponta quase perpendicular ao "fundo do vale" em vez de ao longo dele — o algoritmo dá passos em ziguezague, avançando pouco a cada iteração. O número de iterações necessárias para convergir cresce com κ : problemas mal condicionados são lentos **mesmo com a melhor taxa de aprendizado possível**.

Isso conecta diretamente com features de escalas muito diferentes: uma feature em milhares e outra entre 0 e 1 produzem uma Hessiana mal condicionada, e é por isso que **padronizar features acelera o treinamento** de praticamente qualquer modelo treinado por gradiente — não é só uma boa prática de pré-processamento, é uma correção direta do número de condição do problema de otimização.

NO MERCADO

Pré-condicionamento — transformar o problema para reduzir κ antes de otimizar — é a ideia por trás de normalizações em redes neurais (batch norm, layer norm) e por trás de otimizadores de segunda ordem (que usam a Hessiana, ou uma aproximação dela, para corrigir a direção do passo). Adam pode ser visto como uma forma barata de pré-condicionamento diagonal.

9. Erros que custam caro — checklist

- Escolher a taxa de aprendizado sem observar a curva de perda — se ela oscila ou explode, η está grande demais.
- Não padronizar features antes de treinar um modelo por gradiente — piora o condicionamento e desacelera (ou impede) a convergência.
- Confundir "a perda parou de cair" com "convergiu para o ótimo global" — em problemas não-convexos, pode ser um mínimo local, uma sela, ou um platô.
- Usar gradiente descendente em lote (full-batch) com datasets grandes, quando mini-batches seriam ordens de magnitude mais rápidos por passo.
- Esquecer que a derivada numérica exige escolher h com cuidado — nem grande demais (viés), nem pequeno demais (ruído de ponto flutuante).
- Tratar "não-convexo" como sinônimo de "sem esperança" — redes neurais treinam bem na prática apesar da não-convexidade da perda.

10. Para ir além

- Boyd & Vandenberghe, *Convex Optimization* — o texto de referência, disponível gratuitamente pelos autores.
- Nocedal & Wright, *Numerical Optimization* — cobertura completa de métodos de primeira e segunda ordem.
- Ruder, *An overview of gradient descent optimization algorithms* — o levantamento mais citado sobre SGD, momentum, Adagrad, RMSprop e Adam.
- 3Blue1Brown, *Essence of Calculus* e o vídeo sobre gradiente descendente em redes neurais — a intuição visual antes da álgebra.